Contoh Jawaban Lengkap + Kode: LEC Datnal

Dataset contoh, kode Pandas/NumPy yang sudah dijalankan, dan interpretasi untuk Q1–Q3 (mengacu pada rubrik dosen)

Catatan penting: kolom dan isi dataset di bawah ini saya buat sendiri sebagai ilustrasi, karena dataset asli dosen belum diketahui. Strukturnya mengikuti rubrik: dua dataset, yaitu Survey (dengan ResponseID) dan data perilaku pembelian. Semua kode sudah dijalankan dan output yang tampil adalah hasil aslinya. Saat ujian, ganti nama kolom dengan kolom dataset dosen; logika jawabannya sama. Sampelnya sangat kecil (13 responden, 22 transaksi), jadi angkanya untuk mencontohkan cara menjawab, bukan temuan yang bisa digeneralisasi.

A. Cuplikan Dataset Contoh

Dataset 1: Survey (satu baris = satu responden). Sel berwarna menandai masalah kualitas data yang sengaja ditanam: merah muda = nilai kosong, kuning = baris duplikat, oranye = calon outlier.

ResponseID	Age	Gender	City	Monthly_Income	Satisfaction	Preferred_Channel
R001	25	Female	Jakarta	8.500.000	4	App
R002	37	Male	Bandung	12.000.000	3	Website
R003	29	Female	Surabaya	9.000.000	5	App
R004	41	Male	Jakarta	(kosong)	2	Website
R005	33	Female	Bali	10.500.000	4	App
R005	33	Female	Bali	10.500.000	4	App
R006	(kosong)	Male	Medan	15.000.000	3	Website
R007	26	Female	Jakarta	7.500.000	5	App
R008	38	Male	(kosong)	11.000.000	2	Website
R009	31	Female	Surabaya	9.500.000	4	App
R010	27	Male	Bali	8.000.000	(kosong)	App
R011	45	Male	Jakarta	95.000.000	3	Website
R012	24	Female	Bandung	6.500.000	5	App
R013	34	Female	Medan	9.200.000	3	App

Dataset 2: Purchases (satu baris = satu transaksi; 8 dari 22 baris ditampilkan). Dihubungkan ke Survey lewat **ResponseID** (satu responden bisa punya banyak transaksi).

OrderID	ResponseID	Order_Date	Category	Quantity	Unit_Price	Payment
O001	R001	2026-08-03	Fashion	2	150.000	E-Wallet
O002	R001	2026-08-21	Beauty	1	90.000	E-Wallet
O003	R001	2026-09-14	Fashion	1	250.000	Credit Card
O004	R002	2026-07-12	Electronics	1	1.200.000	Credit Card
O005	R003	2026-08-05	Beauty	3	85.000	E-Wallet
O006	R003	2026-08-19	Fashion	2	180.000	E-Wallet
O007	R003	2026-09-02	Beauty	2	95.000	E-Wallet
O008	R003	2026-09-28	Fashion	1	300.000	E-Wallet
...						

Masalah kualitas data yang ditanam: baris duplikat (R005), 4 sel kosong (Age R006, City R008, Monthly_Income R004, Satisfaction R010), outlier pendapatan (R011 = Rp95.000.000), responden tanpa transaksi (R013), dan Order_Date bertipe string. Kode lengkap pembuatan kedua dataset ada di Lampiran.

B. Q1: 5V Big Data dan CRISP-DM (BU, DA, DP)

Pertanyaan latihan: jelaskan 5V Big Data dan terapkan tahap Business Understanding, Data Understanding, dan Data Preparation CRISP-DM pada analisis perilaku pembelian dengan dua dataset tersebut.

Tujuan analisis: memahami perilaku pembelian pelanggan dan mengidentifikasi pelanggan yang berisiko berhenti berbelanja, agar strategi promosi lebih tepat sasaran.

Q1a. 5V dalam konteks dataset

V	Penjelasan	Penerapan pada Survey + Purchases	Tantangan → pendekatan
Volume	Jumlah data yang besar	Transaksi bertambah setiap hari dan satu responden menghasilkan banyak baris di Purchases, sehingga Purchases jauh lebih besar daripada Survey.	Proses lambat → database/cloud, pemrosesan per bagian (chunk)
Velocity	Kecepatan data dibuat dan diproses	Transaksi masuk terus dari app dan website; survei masuk berkala.	Data cepat usang → pembaruan otomatis (batch harian atau streaming)
Variety	Ragam tipe dan sumber data	Numerik (Monthly_Income, Quantity), kategorikal (City, Category), datetime (Order_Date); jawaban teks bebas pada survei (jika ada) bersifat tidak terstruktur.	Format berbeda → konversi tipe, encoding, merge dengan ResponseID
Veracity	Keakuratan dan keandalan data	Ada duplikat, nilai kosong, outlier pendapatan, dan tanggal bertipe string.	Data kotor → cleaning, imputasi, deteksi outlier, validasi
Value	Manfaat bisnis dari data	Segmentasi pelanggan dan deteksi pelanggan at-risk sehingga promosi tidak terbuang pada pelanggan yang salah.	Analisis tidak terarah → rumuskan tujuan dan KPI di Business Understanding

Q1b. CRISP-DM: tiga tahap awal

Tahap	Tujuan	Kriteria sukses terukur	Aktivitas untuk kedua dataset
Business Understanding	Merumuskan masalah: promosi kurang tepat sasaran dan loyalitas sulit dipantau; tujuan: mengenali pola belanja dan pelanggan at-risk.	1) Segmentasi menghasilkan ≥ 3 segmen dengan perilaku berbeda. 2) Pelanggan at-risk terdeteksi dengan recall ≥ 80%.	Tentukan pertanyaan bisnis, KPI (frekuensi, recency, nilai belanja), stakeholder (pemasaran, produk), dan kolom yang dibutuhkan dari Survey dan Purchases.
Data Understanding	Memahami struktur, kualitas, dan pola awal data.	1) 100% kolom terdokumentasi tipe dan jumlah kosongnya. 2) ≥ 95% ResponseID di Purchases cocok dengan Survey.	*dtypes*, *isnull().sum()*, *duplicated()*, *describe()*, cek kecocokan key dengan set, visualisasi awal.
Data Preparation	Menghasilkan clean dataset siap modeling.	1) 0 missing dan 0 duplikat setelah preparation. 2) Tipe kolom sesuai dan ≥ 3 fitur baru (recency, frequency, monetary) terbentuk.	*drop_duplicates*, *fillna*, IQR, *to_datetime*, *merge*, feature creation, encoding, scaling (lihat bagian C dan D).

C. Q2: Preprocessing, Struktur Data Python, Pandas/NumPy

Pertanyaan latihan: jelaskan tahapan preprocessing, struktur data Python (list, dictionary, set), dan operasi Pandas/NumPy untuk EDA pada kedua dataset.

C1. Preprocessing

Langkah 1: identifikasi kualitas data. Tujuannya mengetahui masalah sebelum mengubah apa pun.

KODE

```
print(survey.dtypes)                      # tipe data tiap kolom
print()
print("Missing values per kolom:")
print(survey.isnull().sum())              # jumlah nilai kosong
print()
print("Baris duplikat:", survey.duplicated().sum())
print(survey[survey.duplicated(keep=False)].to_string())   # tampilkan baris yang terduplikasi
```

OUTPUT (hasil asli saat kode dijalankan)

```
ResponseID              str
Age                 float64
Gender                  str
City                    str
Monthly_Income      float64
Satisfaction        float64
Preferred_Channel       str
dtype: object

Missing values per kolom:
ResponseID          0
Age                 1
Gender              0
City                1
Monthly_Income      1
Satisfaction        1
Preferred_Channel   0
dtype: int64

Baris duplikat: 1
  ResponseID   Age  Gender  City  Monthly_Income  Satisfaction Preferred_Channel
4       R005  33.0  Female  Bali      10500000.0           4.0               App
5       R005  33.0  Female  Bali      10500000.0           4.0               App
```

Hasil: empat kolom masing-masing punya 1 nilai kosong (Age, City, Monthly_Income, Satisfaction), ada 1 baris duplikat (R005), dan Age bertipe float karena adanya NaN.

Langkah 2–3: cleaning, imputasi, dan konversi tipe.

KODE

```
# 1) Hapus duplikat
survey = survey.drop_duplicates().reset_index(drop=True)

# 2) Imputasi missing values
survey["Age"] = survey["Age"].fillna(survey["Age"].median())                  # numerik -> median
survey["Monthly_Income"] = survey["Monthly_Income"].fillna(survey["Monthly_Income"].median())
survey["Satisfaction"] = survey["Satisfaction"].fillna(survey["Satisfaction"].median())
survey["City"] = survey["City"].fillna(survey["City"].mode()[0])              # kategorikal -> modus

# 3) Konversi tipe data
survey["Age"] = survey["Age"].astype(int)
survey["Satisfaction"] = survey["Satisfaction"].astype(int)
purchases["Order_Date"] = pd.to_datetime(purchases["Order_Date"])             # string -> datetime

# 4) Kolom turunan di purchases
purchases["Total_Price"] = purchases["Quantity"] * purchases["Unit_Price"]

# 5) Validasi ulang
print("Sisa missing :", survey.isnull().sum().sum())
print("Sisa duplikat:", survey.duplicated().sum())
print("Jumlah baris :", len(survey))
print(purchases.dtypes[["Order_Date", "Total_Price"]])
```

OUTPUT (hasil asli saat kode dijalankan)

```
Sisa missing : 0
Sisa duplikat: 0
Jumlah baris : 13
Order_Date     datetime64[us]
Total_Price             int64
dtype: object
```

Teknik	Alasan pemilihan
drop_duplicates()	Duplikat membuat satu responden terhitung dua kali dan membiaskan rata-rata.
Median untuk Age, Monthly_Income, Satisfaction	Data numerik; median tidak terpengaruh nilai ekstrem (pendapatan punya outlier).
Modus untuk City	Data kategorikal tidak punya rata-rata; modus adalah nilai paling umum.
pd.to_datetime()	Agar selisih hari (recency) dan filter periode bisa dihitung.
astype(int)	Age dan Satisfaction bulat setelah tidak ada lagi NaN.
Total_Price = Quantity × Unit_Price	Nilai tiap transaksi diperlukan untuk analisis penjualan.

Tips versi Pandas: soal aslab menulis *df['kolom'].fillna(x, inplace=True)*. Pada Pandas 3.x cara ini tidak lagi mengubah DataFrame asli (muncul ChainedAssignmentError). Gunakan penugasan ulang seperti di kode atas: *df['kolom'] = df['kolom'].fillna(x)*.

Langkah 4: deteksi dan penanganan outlier (IQR).

KODE

```
q1 = survey["Monthly_Income"].quantile(0.25)
q3 = survey["Monthly_Income"].quantile(0.75)
iqr = q3 - q1
lower, upper = q1 - 1.5 * iqr, q3 + 1.5 * iqr
print(f"Q1={q1:,.0f}  Q3={q3:,.0f}  IQR={iqr:,.0f}")
print(f"Batas bawah={lower:,.0f}  Batas atas={upper:,.0f}")

outliers = survey[(survey["Monthly_Income"] < lower) | (survey["Monthly_Income"] > upper)]
print()
print(outliers[["ResponseID", "Monthly_Income"]].to_string())

# Penanganan: winsorizing (nilai ekstrem diganti batas atas), data tidak dibuang
survey["Income_Clean"] = survey["Monthly_Income"].clip(lower=lower, upper=upper)
print()
print("Income maksimum sebelum:", f"{survey['Monthly_Income'].max():,.0f}")
print("Income maksimum sesudah:", f"{survey['Income_Clean'].max():,.0f}")
```

OUTPUT (hasil asli saat kode dijalankan)

```
Q1=8,500,000  Q3=11,000,000  IQR=2,500,000
Batas bawah=4,750,000  Batas atas=14,750,000

   ResponseID  Monthly_Income
5        R006      15000000.0
10       R011      95000000.0

Income maksimum sebelum: 95,000,000
Income maksimum sesudah: 14,750,000
```

Hasil: dua nilai di luar batas, yaitu R006 (Rp15 juta, sedikit di atas batas) dan R011 (Rp95 juta, sangat ekstrem). Dipilih *winsorizing* (nilai ekstrem diganti batas atas), bukan dihapus, karena datanya hanya 13 baris dan R011 adalah pelanggan nyata dengan belanja terbesar.

C2. Struktur data Python (list, dictionary, set, tuple)

KODE

```
# LIST: nilai unik kategori produk (berurutan, bisa diubah, boleh duplikat)
category_list = purchases["Category"].unique().tolist()
print("List     :", category_list)

# DICTIONARY: kategori -> rata-rata nilai transaksi (pasangan key:value)
avg_by_cat = purchases.groupby("Category")["Total_Price"].mean().round(0).to_dict()
print("Dict     :", avg_by_cat)
print("Akses key:", f"Rp{avg_by_cat['Beauty']:,.0f}")

# SET: unik & mendukung operasi himpunan -> cek ResponseID di kedua dataset
id_survey = set(survey["ResponseID"])
id_buy = set(purchases["ResponseID"])
print("Set irisan (survey & beli)   :", len(id_survey & id_buy), "responden")
print("Set selisih (survey tanpa beli):", id_survey - id_buy)

# TUPLE: data tetap yang tidak boleh berubah (mis. ambang segmentasi)
SEGMENT_LIMITS = (500_000, 1_500_000)      # batas Low/Medium/High spender
print("Tuple    :", SEGMENT_LIMITS)
```

OUTPUT (hasil asli saat kode dijalankan)

```
List     : ['Fashion', 'Beauty', 'Electronics', 'Home']
Dict     : {'Beauty': 180714.0, 'Electronics': 2625000.0, 'Fashion': 276111.0, 'Home': 525000.0}
Akses key: Rp180,714
Set irisan (survey & beli)   : 12 responden
Set selisih (survey tanpa beli): {'R013'}
Tuple    : (500000, 1500000)
```

Struktur	Alasan dipakai pada data ini
List	Menyimpan daftar kategori produk secara berurutan, mis. untuk perulangan atau pilihan filter dashboard.
Dictionary	Pencarian cepat kategori → rata-rata transaksi; juga untuk mapping kolom, mis. {'Female': 0, 'Male': 1}.
Set	Otomatis unik dan mendukung irisan/selisih: terbukti 12 responden punya transaksi dan R013 tidak.
Tuple	Menyimpan nilai tetap (ambang segmentasi) yang tidak boleh berubah tanpa sengaja.

Insight: rata-rata nilai transaksi tertinggi ada di Electronics (Rp2.625.000), terendah di Beauty (Rp180.714).

C3. Operasi Pandas/NumPy untuk EDA

KODE

```
# 1) merge(): gabungkan survei + transaksi lewat ResponseID
merged = purchases.merge(survey, on="ResponseID", how="left")
print("1) merge          :", merged.shape)

# 2) groupby().agg(): ringkasan penjualan per kategori
sales_cat = (purchases.groupby("Category")["Total_Price"]
             .agg(["count", "sum", "mean"]).sort_values("sum", ascending=False))
print("\n2) groupby().agg():")
print(sales_cat.to_string())

# 3) value_counts(): metode pembayaran terbanyak
print("\n3) value_counts():")
print(purchases["Payment"].value_counts().to_string())

# 4) np.where(): label bersyarat (belanja besar / kecil)
merged["Order_Size"] = np.where(merged["Total_Price"] >= 500_000, "Large", "Small")
print("\n4) np.where       :", merged["Order_Size"].value_counts().to_dict())

# 5) np.percentile(): batas 75% nilai transaksi
p75 = np.percentile(purchases["Total_Price"], 75)
print("\n5) np.percentile  : P75 nilai transaksi = Rp" + f"{p75:,.0f}")
```

OUTPUT (hasil asli saat kode dijalankan)

```
1) merge          : (22, 15)

2) groupby().agg():
             count       sum          mean
Category
Electronics      4  10500000  2.625000e+06
Fashion          9   2485000  2.761111e+05
Beauty           7   1265000  1.807143e+05
Home             2   1050000  5.250000e+05

3) value_counts():
Payment
E-Wallet       15
Credit Card     5
Transfer        2

4) np.where       : {'Small': 17, 'Large': 5}

5) np.percentile  : P75 nilai transaksi = Rp427,500
```

Operasi	Tujuan	Hasil pada data
merge()	Menggabungkan Purchases dan Survey lewat ResponseID	22 baris × 15 kolom
groupby().agg()	Meringkas penjualan per kategori	Electronics Rp10,5 juta dari 4 transaksi; Fashion paling sering (9 transaksi)
value_counts()	Mengetahui metode pembayaran terbanyak	E-Wallet 15 dari 22 transaksi (sekitar 68%)
np.where()	Membuat label bersyarat tanpa perulangan	5 transaksi Large (≥ Rp500.000), 17 Small
np.percentile()	Menentukan batas persentil nilai transaksi	75% transaksi bernilai ≤ Rp427.500

D. Q3: Feature Creation, Visualisasi, Interpretasi EDA

Pertanyaan latihan: buat minimal 2 fitur baru dari informasi yang ada, berikan minimal 2 visualisasi yang cocok untuk stakeholder, lalu interpretasikan hasil EDA.

D1. Feature creation

KODE

```
REF_DATE = pd.Timestamp("2026-10-01")      # tanggal acuan analisis

# Agregasi transaksi -> satu baris per pelanggan
cust = purchases.groupby("ResponseID").agg(
    total_spent=("Total_Price", "sum"),            # Monetary
    n_orders=("OrderID", "count"),                 # Frequency
    last_order=("Order_Date", "max"),
).reset_index()

# Fitur baru 1-3: avg_order_value, recency_days
cust["avg_order_value"] = (cust["total_spent"] / cust["n_orders"]).round(0)
cust["recency_days"] = (REF_DATE - cust["last_order"]).dt.days      # Recency

# Gabungkan ke survei (left join: responden tanpa transaksi tetap ikut)
df = survey.merge(cust, on="ResponseID", how="left")
df[["total_spent", "n_orders"]] = df[["total_spent", "n_orders"]].fillna(0)

# Fitur baru 4: age_group (binning dengan pd.cut)
df["age_group"] = pd.cut(df["Age"], bins=[0, 25, 35, 100],
                         labels=["<=25", "26-35", ">35"])

# Fitur baru 5: income_level (binning dengan pd.qcut -> 3 kelompok sama besar)
df["income_level"] = pd.qcut(df["Income_Clean"], q=3, labels=["Low", "Mid", "High"])

# Fitur baru 6: customer_status (np.where): aktif / berisiko churn
df["customer_status"] = np.where(df["recency_days"].isna() | (df["recency_days"] > 45),
                                 "At-risk", "Active")

cols = ["ResponseID", "Age", "age_group", "income_level", "total_spent",
        "n_orders", "avg_order_value", "recency_days", "customer_status"]
print(df[cols].to_string())
```

OUTPUT (hasil asli saat kode dijalankan)

```
   ResponseID  Age age_group income_level  total_spent  n_orders  avg_order_value  recency_days customer_status
0        R001   25      <=25          Low     640000.0       3.0         213333.0          17.0          Active
1        R002   37       >35         High    1200000.0       1.0        1200000.0          81.0         At-risk
2        R003   29     26-35          Low    1105000.0       4.0         276250.0           3.0          Active
3        R004   41       >35          Mid     450000.0       1.0         450000.0         103.0         At-risk
4        R005   33     26-35          Mid     420000.0       2.0         210000.0          22.0          Active
5        R006   32     26-35         High    2500000.0       1.0        2500000.0          63.0         At-risk
6        R007   26     26-35          Low     705000.0       3.0         235000.0          11.0          Active
7        R008   38       >35         High     600000.0       1.0         600000.0          88.0         At-risk
8        R009   31     26-35          Mid     440000.0       2.0         220000.0          13.0          Active
9        R010   27     26-35          Low     800000.0       1.0         800000.0          26.0          Active
10       R011   45       >35         High    6000000.0       1.0        6000000.0          75.0         At-risk
11       R012   24      <=25          Low     440000.0       2.0         220000.0           6.0          Active
12       R013   34     26-35          Mid           0.0       0.0              NaN           NaN         At-risk
```

Fitur baru	Cara membuat	Kaitan dengan perilaku pembelian
recency_days	Selisih tanggal acuan dengan order terakhir	Seberapa baru pelanggan belanja; indikator risiko churn
n_orders (frequency)	*count* transaksi per ResponseID	Tingkat loyalitas/keseringan belanja
total_spent (monetary)	*sum* Total_Price per ResponseID	Nilai pelanggan bagi bisnis
avg_order_value	total_spent ÷ n_orders	Pola ukuran belanja (sering-kecil atau jarang-besar)
age_group	*pd.cut* pada Age	Segmentasi pemasaran berdasarkan usia
income_level	*pd.qcut* pada Income_Clean	Segmentasi daya beli yang tidak terpengaruh outlier
customer_status	*np.where*: recency > 45 hari atau belum pernah beli	Label sederhana pelanggan at-risk

Catatan: (1) ambang 45 hari adalah asumsi; sesuaikan dengan siklus pembelian bisnisnya. (2) *left join* membuat R013 (tanpa transaksi) tetap ikut: total_spent dan n_orders diisi 0, recency dibiarkan kosong, dan statusnya At-risk.

D2. EDA: perbedaan kelompok, hubungan, dan anomali

KODE

```
# (a) Perbedaan antar kelompok: channel
print("Rata-rata total belanja per channel:")
print(df.groupby("Preferred_Channel")[["total_spent", "n_orders", "Satisfaction"]].mean().round(1).to_string())

# (b) Perbedaan antar kelompok: kelompok usia
print("\nRata-rata total belanja per kelompok usia:")
print(df.groupby("age_group", observed=True)["total_spent"].agg(["count", "mean"]).round(0).to_string())

# (c) Hubungan antar variabel (korelasi)
num_cols = ["Age", "Income_Clean", "Satisfaction", "total_spent", "n_orders"]
corr = df[num_cols].corr().round(2)
print("\nKorelasi:")
print(corr.to_string())

# (d) Anomali: pelanggan berisiko churn
print("\nPelanggan At-risk:")
print(df.loc[df["customer_status"] == "At-risk",
             ["ResponseID", "total_spent", "recency_days", "Satisfaction"]].to_string())
```

OUTPUT (hasil asli saat kode dijalankan)

```
Rata-rata total belanja per channel:
                  total_spent  n_orders  Satisfaction
Preferred_Channel
App                  568750.0       2.1           4.1
Website             2150000.0       1.0           2.6

Rata-rata total belanja per kelompok usia:
           count       mean
age_group
<=25           2   540000.0
26-35          7   852857.0
>35            4  2062500.0

Korelasi:
               Age  Income_Clean  Satisfaction  total_spent  n_orders
Age           1.00          0.70         -0.73         0.54     -0.57
Income_Clean  0.70          1.00         -0.50         0.74     -0.42
Satisfaction -0.73         -0.50          1.00        -0.17      0.80
total_spent   0.54          0.74         -0.17         1.00     -0.16
n_orders     -0.57         -0.42          0.80        -0.16      1.00

Pelanggan At-risk:
   ResponseID  total_spent  recency_days  Satisfaction
1        R002    1200000.0          81.0             3
3        R004     450000.0         103.0             2
5        R006    2500000.0          63.0             3
7        R008     600000.0          88.0             2
10       R011    6000000.0          75.0             3
12       R013          0.0           NaN             3
```

D3. Visualisasi untuk stakeholder

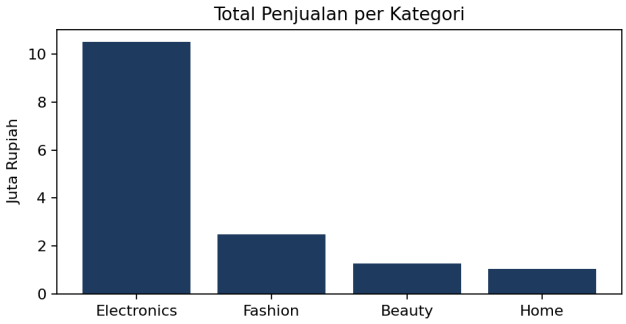
KODE

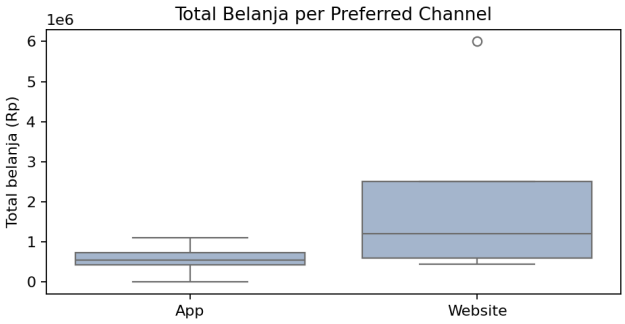
```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import seaborn as sns

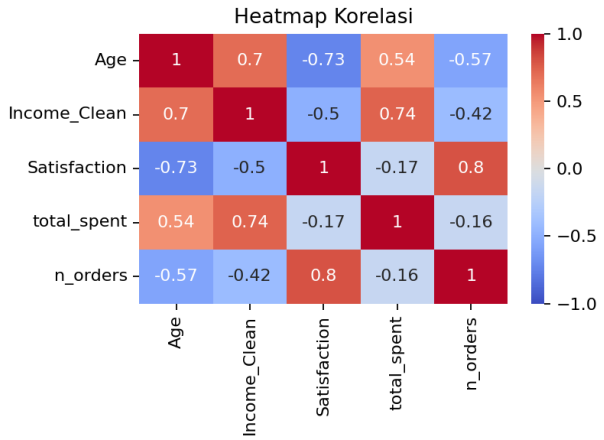
# Visualisasi 1: Bar chart - total penjualan per kategori (untuk manajer produk)
fig, ax = plt.subplots(figsize=(6, 3.2))
sales = purchases.groupby("Category")["Total_Price"].sum().sort_values(ascending=False)
ax.bar(sales.index, sales.values / 1e6, color="#1F3A5F")
ax.set_title("Total Penjualan per Kategori")
ax.set_ylabel("Juta Rupiah")
plt.tight_layout(); plt.savefig("fig_bar.png", dpi=160); plt.close()

# Visualisasi 2: Boxplot - total belanja per channel (untuk tim pemasaran)
fig, ax = plt.subplots(figsize=(6, 3.2))
sns.boxplot(data=df, x="Preferred_Channel", y="total_spent", color="#9DB4D3", ax=ax)
ax.set_title("Total Belanja per Preferred Channel")
ax.set_xlabel(""); ax.set_ylabel("Total belanja (Rp)")
plt.tight_layout(); plt.savefig("fig_box.png", dpi=160); plt.close()

# Visualisasi 3: Heatmap korelasi (untuk tim data)
fig, ax = plt.subplots(figsize=(5.2, 3.8))
sns.heatmap(corr, annot=True, cmap="coolwarm", vmin=-1, vmax=1, ax=ax)
ax.set_title("Heatmap Korelasi")
plt.tight_layout(); plt.savefig("fig_heat.png", dpi=160); plt.close()
print("3 gambar tersimpan")
```







Heatmap korelasi (untuk tim data)

Total belanja paling berkaitan dengan pendapatan (0,74): semakin tinggi pendapatan, semakin besar belanja. Kepuasan berkaitan kuat dengan *frekuensi* belanja (0,80) tetapi tidak dengan *nilai* belanja (−0,17). Age dan Income saling berkorelasi (0,70), sehingga berpotensi redundan bila keduanya dipakai di model.

Visualisasi	Mengapa dipilih	Stakeholder
Bar chart penjualan per kategori	Membandingkan kategori secara langsung	Manajer produk
Boxplot total belanja per channel	Membandingkan sebaran dan outlier antar kelompok	Tim pemasaran
Heatmap korelasi	Melihat hubungan antar variabel sekaligus	Tim data / analis

D4. Interpretasi EDA dan keputusan bisnis

Urutan menjawab: **pola → perbedaan → anomali → hubungan → keputusan**.

- **Pola dan perbedaan (channel):** pengguna Website rata-rata belanja Rp2.150.000 tetapi hanya 1 transaksi dan kepuasan 2,6; pengguna App rata-rata Rp568.750 dengan 2,1 transaksi dan kepuasan 4,1. Jadi Website berisi pembeli sekali-besar yang kurang puas, sedangkan App berisi pembeli sering-kecil yang puas.
- **Perbedaan (usia):** rata-rata total belanja ≤25 tahun Rp540.000 (n=2), 26–35 tahun Rp852.857 (n=7), >35 tahun Rp2.062.500 (n=4).
- **Anomali:** seluruh 5 responden pengguna Website berstatus At-risk (recency 63–103 hari); R011 berpendapatan Rp95 juta sekaligus belanja terbesar (Rp6.000.000); R013 belum pernah membeli.
- **Konsentrasi pendapatan:** Electronics menyumbang sekitar 69% penjualan dari hanya 4 transaksi, dan dua pelanggan (R011 dan R006) menyumbang Rp8,5 juta, lebih dari separuh total penjualan Rp15,3 juta.

Keputusan yang disarankan:

- Program reaktivasi (voucher, perbaikan pengalaman website) dengan prioritas pelanggan bernilai tinggi yang at-risk: R011, R006, R002.
- Pengguna App: cross-sell Fashion dan Beauty serta program loyalitas untuk mempertahankan frekuensi belanja.
- Kurangi ketergantungan pada sedikit pelanggan Electronics dengan memperluas basis pembeli kategori tersebut.

Batasan: n = 13 responden, korelasi tidak membuktikan sebab-akibat, dan ambang at-risk 45 hari adalah asumsi. Sebutkan batasan seperti ini di jawaban ujian agar interpretasi terlihat hati-hati.

D5. Tambahan: encoding dan scaling sebelum modeling

KODE

```
from sklearn.preprocessing import StandardScaler

# One-hot encoding (variabel nominal: tidak ada urutan)
model_df = pd.get_dummies(df[["Gender", "Preferred_Channel"]], drop_first=True, dtype=int)

# Scaling (StandardScaler) agar skala fitur seragam
scaler = StandardScaler()
model_df[["Age_s", "Income_s", "Spent_s"]] = scaler.fit_transform(
    df[["Age", "Income_Clean", "total_spent"]]).round(2)

print(model_df.head(5).to_string())
```

OUTPUT (hasil asli saat kode dijalankan)

```
   Gender_Male  Preferred_Channel_Website  Age_s  Income_s  Spent_s
0            0                         0  -1.21     -0.63    -0.36
1            1                         1   0.73      0.80     0.02
2            0                         0  -0.56     -0.43    -0.05
3            1                         1   1.38     -0.28    -0.48
4            0                         0   0.09      0.19    -0.50
```

One-hot encoding dipakai untuk variabel nominal (Gender, Preferred_Channel) karena tidak punya urutan. Untuk variabel ordinal (mis. Membership_Level: Basic < Silver < Gold) cukup label encoding atau *map* berurutan. **StandardScaler** menyamakan skala fitur (usia puluhan, pendapatan jutaan) agar tidak ada fitur yang mendominasi model.

E. Checklist Sebelum Menjawab (sesuai rubrik)

- **Case application:** sebut nama dataset dan kolom dari soal di setiap jawaban (Survey, ResponseID, perilaku pembelian).
- **Setiap istilah:** tulis fungsi, alasan, dan contoh, jangan hanya menyebut nama.
- **Q1:** kelima V dengan contoh dan pendekatan; BU, DA, DP dengan tujuan, ≥ 2 kriteria sukses terukur, dan aktivitas untuk kedua dataset.

- **Q2:** langkah preprocessing lengkap; list, dictionary, set dengan contoh; ≥ 3 operasi Pandas/NumPy beserta tujuannya.
- **Q3:** ≥ 2 fitur baru; ≥ 2 visualisasi beserta alasan dan stakeholder; interpretasi pola, anomali, hubungan, perbedaan, lalu keputusan.
- **Gaya penulisan:** ringkas, terstruktur (poin atau tabel), bahasa akademik.

Lampiran: Kode Pembuatan Dataset Contoh

Salin kode ini untuk menjalankan semua contoh di atas dari awal (urutan kode: Lampiran, lalu C1, C2, C3, D1, D2, D3, D5).

KODE

```
import pandas as pd
import numpy as np

# Dataset 1: Survey (satu baris = satu responden)
survey = pd.DataFrame({
    "ResponseID":        ["R001","R002","R003","R004","R005","R005","R006",
                          "R007","R008","R009","R010","R011","R012","R013"],
    "Age":               [25, 37, 29, 41, 33, 33, np.nan, 26, 38, 31, 27, 45, 24, 34],
    "Gender":            ["Female","Male","Female","Male","Female","Female","Male",
                          "Female","Male","Female","Male","Male","Female","Female"],
    "City":              ["Jakarta","Bandung","Surabaya","Jakarta","Bali","Bali","Medan",
                          "Jakarta",np.nan,"Surabaya","Bali","Jakarta","Bandung","Medan"],
    "Monthly_Income":    [8500000, 12000000, 9000000, np.nan, 10500000, 10500000, 15000000,
                          7500000, 11000000, 9500000, 8000000, 95000000, 6500000, 9200000],
    "Satisfaction":      [4, 3, 5, 2, 4, 4, 3, 5, 2, 4, np.nan, 3, 5, 3],
    "Preferred_Channel": ["App","Website","App","Website","App","App","Website",
                          "App","Website","App","App","Website","App","App"],
})

# Dataset 2: Purchases (satu baris = satu transaksi), dihubungkan lewat ResponseID
purchases = pd.DataFrame({
    "OrderID":    ["O%03d" % i for i in range(1, 23)],
    "ResponseID": ["R001","R001","R001","R002","R003","R003","R003","R003","R004","R005",
                   "R005","R006","R007","R007","R007","R008","R009","R009","R010","R011",
                   "R012","R012"],
    "Order_Date": ["2026-08-03","2026-08-21","2026-09-14","2026-07-12","2026-08-05","2026-08-19",
                   "2026-09-02","2026-09-28","2026-06-20","2026-08-10","2026-09-09","2026-07-30",
                   "2026-08-14","2026-09-01","2026-09-20","2026-07-05","2026-08-25","2026-09-18",
                   "2026-09-05","2026-07-18","2026-08-30","2026-09-25"],
    "Category":   ["Fashion","Beauty","Fashion","Electronics","Beauty","Fashion","Beauty","Fashion",
                   "Home","Fashion","Beauty","Electronics","Fashion","Beauty","Fashion","Home",
                   "Beauty","Fashion","Electronics","Electronics","Beauty","Fashion"],
    "Quantity":   [2,1,1,1,3,2,2,1,1,1,2,1,2,1,1,2,2,1,1,2,3,1],
    "Unit_Price": [150000,90000,250000,1200000,85000,180000,95000,300000,450000,220000,100000,
                   2500000,160000,110000,275000,300000,90000,260000,800000,3000000,80000,200000],
    "Payment":    ["E-Wallet","E-Wallet","Credit Card","Credit Card","E-Wallet","E-Wallet","E-Wallet",
                   "E-Wallet","Transfer","E-Wallet","E-Wallet","Credit Card","E-Wallet","E-Wallet",
                   "E-Wallet","Transfer","E-Wallet","Credit Card","E-Wallet","Credit Card",
                   "E-Wallet","E-Wallet"],
})

print("Survey   :", survey.shape)
print("Purchases:", purchases.shape)
print(survey.head(6).to_string())
print()
print(purchases.head(5).to_string())
```

OUTPUT (hasil asli saat kode dijalankan)

```
Survey   : (14, 7)
Purchases: (22, 7)
  ResponseID   Age  Gender      City  Monthly_Income  Satisfaction Preferred_Channel
0       R001  25.0  Female   Jakarta       8500000.0           4.0               App
1       R002  37.0    Male   Bandung      12000000.0           3.0           Website
2       R003  29.0  Female  Surabaya       9000000.0           5.0               App
3       R004  41.0    Male   Jakarta             NaN           2.0           Website
4       R005  33.0  Female      Bali      10500000.0           4.0               App
5       R005  33.0  Female      Bali      10500000.0           4.0               App

  OrderID ResponseID  Order_Date     Category  Quantity  Unit_Price      Payment
0    O001       R001  2026-08-03      Fashion         2      150000     E-Wallet
1    O002       R001  2026-08-21       Beauty         1       90000     E-Wallet
2    O003       R001  2026-09-14      Fashion         1      250000  Credit Card
3    O004       R002  2026-07-12  Electronics         1     1200000  Credit Card
4    O005       R003  2026-08-05       Beauty         3       85000     E-Wallet
```